Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

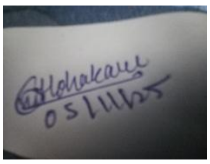
Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Ganesh Pandile
Name and Signature of Student and Date
(Signature and Date in Handwritten)



Session	2025-26 (ODD)	Course Name	Computer vision Lab
Semester	**5**	**Course Code**	**CT**
Roll No	**46**	**Name of Student**	**Ganesh Pandile**

Practical Number	**10**
Course Outcome	**Upon successful completion of the course the students will be able to** 1. Apply image enhancement and smoothing techniques to improve image quality for further analysis. 2. Extract meaningful features from images using descriptors such as HOG and SIFT. 3. Implement and evaluate modern object detection methods including YOLO and R-CNN. 4. Analyze and develop solutions for motion estimation, object recognition, and facial expression recognition using classical and learning-based methods.
Aim	1. Use different filters for image smoothing. 2. Implement parametric model for motion estimation.
Problem Definition	This project aims to develop a Python program that performs image smoothing using various filters to remove noise and enhance image clarity. Additionally, the system will implement a motion estimation technique to detect and visualize movement between two consecutive images, showing how objects have shifted or changed position.
Theory (100 words)	**1.Image Smoothing** Image smoothing removes unwanted noise and softens an image using filters that modify each pixel based on its neighbors. • Average Filter: Replaces each pixel with the mean of all pixels in a kernel, giving a uniform blur. • Gaussian Filter: Applies a Gaussian function to weight nearby pixels more heavily, achieving natural smoothing while keeping edges better. • Median Filter: Replaces each pixel with the median value of its neighborhood, excellent for removing salt-and-pepper noise. • Bilateral Filter: Considers both spatial and intensity similarity to smooth textures while preserving sharp edges and details. **2.Motion Estimation** Motion estimation detects object displacement between consecutive frames. • Shi–Tomasi Corner Detection: Finds strong corner points by

<table>
<tr><td></td><td>analyzing image gradients and selecting those with high minimum eigenvalues (indicating good trackable features).
• Lucas–Kanade Optical Flow: Tracks how these corner points move by assuming brightness constancy and small motion, solving gradient-based equations to estimate displacement vectors.
• The tracked points are then used to compute an Affine Transformation Matrix, representing translation, rotation, and scaling. Finally, arrows are drawn to visualize motion direction and magnitude.</td></tr>
<tr><td>Procedure and Execution

(100 Words)</td><td>Algorithm:
1.Image Filters
1. Read the input image using cv2.imread().
2. Convert the image from BGR to RGB for displaying correctly using Matplotlib.
3. Apply different smoothing filters:
o Average Filter:
Creates a 5×5 kernel and replaces each pixel value with the mean of its neighborhood
Implemented using cv2.blur(img, (5,5)).
o Gaussian Filter:
Weights nearby pixels based on a Gaussian function, reducing high-frequency noise while preserving edges.
Implemented using cv2.GaussianBlur(img, (5,5), 0).
o Median Filter:
Sorts pixel values in the kernel and replaces the center with the median, removing salt-and-pepper noise.
Implemented using cv2.medianBlur(img, 5).
o Bilateral Filter:
Combines spatial and intensity distances to preserve edges while smoothing similar regions.
Implemented using cv2.bilateralFilter(img, 9, 75, 75).
4. Display the original and filtered images using Matplotlib for visual comparison.
5. End.
Key Point:
Average and Gaussian filters are linear (weighted averaging), while Median and Bilateral filters are non-linear (better at preserving edges).

2.Algorithm for Motion Estimation
1. Load two frames from the same video sequence in grayscale using cv2.imread().
2. Detect corner points in the first frame using the Shi–Tomasi Corner Detection algorithm:
o Calculates image gradients in both x and y directions.
o Builds a structure tensor for each pixel and computes eigenvalues.
o Selects points with high minimum eigenvalues (good</td></tr>
</table>

corners).
Implemented using cv2.goodFeaturesToTrack().

3. Track these points in the next frame using the Lucas–Kanade Optical Flow method:
    - Assumes pixel intensity remains constant and motion between frames is small.
    - Solves for displacement (u, v) using image gradients in a local neighborhood.

    Implemented using cv2.calcOpticalFlowPyrLK().
4. Estimate motion transformation using the matched point pairs to find the Affine Transformation Matrix (cv2.estimateAffinePartial2D()), which represents translation, rotation, and scaling.
5. Visualize motion:
    - Draw arrows from old to new feature points using cv2.arrowedLine().
    - Display the original and motion vector images side by side using Matplotlib.
6. End.

Code:
```
import cv2, os
import numpy as np
import matplotlib.pyplot as plt
from tkinter import filedialog, Tk

def choose_image():
    print("\n1. Use image from 'assets'\n2. Upload new image")
    try:
        opt = int(input("Choose option: "))
    except ValueError:
        print("Invalid input.")
        return None

    if opt == 1:
        os.makedirs("assets", exist_ok=True)
        files = [f for f in os.listdir("assets") if
f.lower().endswith(('.jpg', '.jpeg', '.png'))]
        if not files:
            print("No images found in 'assets' folder.")
            return None
        for i, f in enumerate(files, 1):
            print(f"{i}. {f}")
        try:
            idx = int(input("Select image number: ")) - 1
            return os.path.join("assets", files[idx]) if 0 <= idx <
len(files) else None
        except ValueError:
            return None
    elif opt == 2:
        root = Tk(); root.withdraw(); root.attributes('-topmost', True)
        path = filedialog.askopenfilename(title="Select Image",
filetypes=[("Images", "*.jpg *.jpeg *.png")])
```

```
        root.destroy()
        return path if path else None
    return None

def image_smoothing():
    path = choose_image()
    if not path: return
    img = cv2.imread(path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    filters = {
        1: ("Average", cv2.blur(img, (5, 5))),
        2: ("Gaussian", cv2.GaussianBlur(img, (5, 5), 0)),
        3: ("Median", cv2.medianBlur(img, 5)),
        4: ("Bilateral", cv2.bilateralFilter(img, 9, 75, 75))
    }

    print("\n1.Average  2.Gaussian  3.Median  4.Bilateral  5.All")
    try:
        choice = int(input("Choose filter: "))
    except ValueError:
        print("Invalid input."); return

    if choice == 5:
        plt.figure(figsize=(12, 8))
        titles = ["Original"] + [v[0] for v in filters.values()]
        imgs = [img_rgb] + [v[1] for v in filters.values()]
        for i, (title, im) in enumerate(zip(titles, imgs), 1):
            plt.subplot(2, 3, i); plt.imshow(cv2.cvtColor(im,
cv2.COLOR_BGR2RGB))
            plt.title(title); plt.axis('off')
    elif choice in filters:
        plt.figure(figsize=(10, 5))
        plt.subplot(1, 2, 1); plt.imshow(img_rgb); plt.title("Original");
plt.axis('off')
        plt.subplot(1, 2, 2);
plt.imshow(cv2.cvtColor(filters[choice][1],
cv2.COLOR_BGR2RGB))
        plt.title(filters[choice][0]); plt.axis('off')
    else:
        print("Invalid choice."); return
    plt.show()

def motion_estimation():
    folder = "video_frames_output"
    if not os.path.exists(folder):
        print(f"Folder '{folder}' not found!"); return

    frames = sorted([f for f in os.listdir(folder) if
f.lower().endswith(('.jpg', '.png'))])
    if len(frames) < 2:
        print("Need at least 2 frames."); return
```

```
    for i, f in enumerate(frames, 1): print(f"{i}. {f}")
    try:
        f1, f2 = int(input("First frame: ")) - 1, int(input("Second
frame: ")) - 1
        if not (0 <= f1 < len(frames) and 0 <= f2 < len(frames)):
return
    except ValueError:
        print("Invalid input."); return

    img1 = cv2.imread(os.path.join(folder, frames[f1]),
cv2.IMREAD_GRAYSCALE)
    img2 = cv2.imread(os.path.join(folder, frames[f2]),
cv2.IMREAD_GRAYSCALE)
    if img1 is None or img2 is None: print("Error loading frames.");
return

    pts1 = cv2.goodFeaturesToTrack(img1, 100, 0.01, 10)
    pts2, st, _ = cv2.calcOpticalFlowPyrLK(img1, img2, pts1,
None)
    good_old, good_new = pts1[st == 1], pts2[st == 1]
    M, _ = cv2.estimateAffinePartial2D(good_old, good_new)
    print("\nAffine Transformation Matrix:\n", M)

    img_vis = cv2.cvtColor(img1, cv2.COLOR_GRAY2BGR)
    for n, o in zip(good_new, good_old):
        x1, y1 = map(int, o.ravel()); x2, y2 = map(int, n.ravel())
        cv2.arrowedLine(img_vis, (x1, y1), (x2, y2), (0, 255, 0), 1,
tipLength=0.3)

    # Before–After display
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1); plt.imshow(img1, cmap='gray');
plt.title("Before"); plt.axis('off')
    plt.subplot(1, 2, 2); plt.imshow(img_vis[..., ::-1]);
plt.title("Motion Vectors (After)"); plt.axis('off')
    plt.show()

while True:
    print("\n==== MINI PROJECT MENU ====")
    print("1. Image Smoothing\n2. Motion Estimation\n3. Exit")
    try:
        c = int(input("Enter choice: "))
    except ValueError:
        print("Invalid input."); continue

    if c == 1: image_smoothing()
    elif c == 2: motion_estimation()
    elif c == 3: print("Goodbye!"); break
    else: print("Invalid choice.")
```

Output:

```
TERMINAL
PS C:\Users\mithi\5th sem\CvPro> & C:/Users/mithi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/mithi/5th sem/CvPro/Main.py"

==== MINI PROJECT MENU ====
1. Image Smoothing
2. Motion Estimation
3. Exit
Enter choice: 1

1. Use image from 'assets'
2. Upload new image
Choose option: 1
1. bmw-m850i-xdrive-coupe-edition-2020-3840x2160-1418.jpg
2. dope-sukuna-pink-5120x2880-16935.png
3. dramatic-scenery-3840x2160-20802.jpg
4. gta-6-teaser-3840x2160-13559.png
5. imo.jpg
6. macos-big-sur-apple-layers-fluidic-colorful-wwdc-stock-4096x2304-1455.jpg
Select image number: 5

1.Average  2.Gaussian  3.Median  4.Bilateral  5.All
Choose filter: 5
```


```
==== MINI PROJECT MENU ====
1. Image Smoothing
2. Motion Estimation
3. Exit
Enter choice: 2
1. frame_0000.jpg
2. frame_0001.jpg
3. frame_0002.jpg
4. frame_0003.jpg
5. frame_0004.jpg
6. frame_0005.jpg
7. frame_0006.jpg
8. frame_0007.jpg
9. frame_0008.jpg
10. frame_0009.jpg
11. frame_0010.jpg
12. frame_0011.jpg
13. frame_0012.jpg
14. frame_0013.jpg
15. frame_0014.jpg
16. frame_0015.jpg
17. frame_0016.jpg
18. frame_0017.jpg
19. frame_0018.jpg
20. frame_0019.jpg
21. frame_0020.jpg
22. frame_0021.jpg
23. frame_0022.jpg
24. frame_0023.jpg
25. frame_0024.jpg
```

Output Analysis

Image Smoothing:
The program successfully displays the original and filtered images.
- Average Filter: Gives uniform blur, details are lost.
- Gaussian Filter: Smooths naturally and preserves soft edges.
- Median Filter: Removes salt-and-pepper noise, keeps edges sharper.

	• Bilateral Filter: Reduces noise but preserves fine edges. Each filter behaves differently depending on the kernel size and type of noise present. Condition: Works best on noisy or low-contrast images; kernel size affects smoothness. Motion Estimation: The program shows two selected frames with motion vectors (green arrows) indicating movement of tracked corner points. • Shi–Tomasi detects strong corners to track. • Lucas–Kanade tracks their displacement between frames. • The Affine Transformation Matrix shows how the image is shifted, rotated, or scaled. Condition: Requires at least two consecutive frames with detectable texture; motion should be small and consistent for accurate tracking.
Link of student Github profile where lab assignment has	https://github.com/ganeshpandile/Computer-Vision.git

Nagar Yuwak Shikshan Sanstha's

Yeshwantrao Chavan College of Engineering

(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

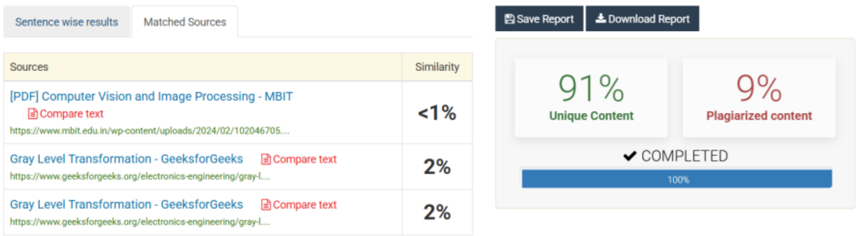
Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

been uploaded	
Conclusion	The project successfully demonstrates fundamental concepts of image processing and computer vision. Image smoothing effectively reduces noise and enhances visual quality, while motion estimation accurately detects and tracks object movement between frames. Together, they showcase how spatial and temporal analysis help in improving image clarity and understanding real-world motion.
Plag Report (Similarity index < 12%)	
Date	8/11/25